Siddhi Jha
A-3

ASSIGNMENT - 1

Aim:
To write a C Program to perform basic matrix operations such as addition, subtraction, multiplication, and transpose of a matrix using two-dimensional arrays.

Objectives:
- To understand the concept of two-dimensional array in C.
- To perform arithmetic operations on matrices.
- To implement matrix addition and subtraction.
- To implement matrix multiplication using nested loops.
- To find the transpose of a matrix.
- To develop programming skills using arrays and loops.

Theory:
A matrix is a rectangular arrangement of elements organized in rows and columns. In C programming, matrices are represented using two-dimensional arrays. Matrix operations are widely used in computer graphics, image processing, machine learning, scientific computing, and many engineering applications.

1) Matrix Addition:
Condition: Both matrices must have the same number of rows & column.
Formula: $C[i][j] = A[i][j] + B[i][j]$

2) Matrix Subtraction:
Condition: Both matrices must have the same dimensions.
Formula: $C[i][j] = A[i][j] - B[i][j]$

3) Matrix Multiplication:

Condition: Number of columns of the first matrix must be equal to the number of rows of second matrix

Formula:

$$C[i][j] = \Sigma(A[i][k] \times B[k][j]) \quad \text{where } k$$

4) Transpose of a Matrix:

The transpose of a matrix is obtained by interchanging its rows and columns.

Formula: $A_T[i][j] = A[j][i]$

These operations are implemented using 2-D arrays, loops, and conditional statements.

FAQs:

1) Explain matrix addition, subtraction, multiplication, and division with suitable examples and state the conditions required for each operation.

Ans: Matrix Addition:

performed by adding the corresponding elements of two matrices

Cond^n: Both matrices must have the same no. of rows and columns

Eg:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \qquad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$A + B = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

Matrix Subtraction:

performed by subtracting corresponding elements.

Cond^n: Both matrices must have same order.

Eg: $A - B = \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix}$

Matrix Multiplication:

Each element of resulting matrix is obtained by multiplying rows of the first matrix with columns of second matrix.

Cond^n: No. of columns of the first matrix must be equal to number of second matrix.

Eg: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $\quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

$$AB = \begin{bmatrix} 5+14 & 6+16 \\ 15+28 & 18+32 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Matrix Transpose:

Obtained by interchanging rows and columns.

Cond^n: Applicable to any matrix.

Eg: $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ $\quad A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$

Describe how 2-D arrays are used to represent matrices in C programming. Explain the process of accessing and manipulating matrix elements using nested loops.

In C progg., matrices are represented using 2-D arrays. A 2-D array stores data in rows & columns similar to a matrix.

Syntax: `int matrix[3][3];`

// here matrix has 3 rows & 3 columns.

Accessing Elements:
Elements are accessed using row and column
Syntax: matrix[1][2];
// this accesses the element in second row & third

Using Nested loops:
Nested loops are used because matrices have rows

Syntax:
```
for (i=0; i<rows; i++)
{
    for(j=0; j<cols; j++)
    {
        scanf("%d", &matrix[i][j]);
    }
}
// outer loop controls rows
// inner loop controls columns.
```

3) Write the algorithm for performing matrix multiplication and explain why compatibility between matrices is re before multiplication.

ns:
Algorithm:
S-1: START
S-2: Input rows & columns of Matrix A and Matrix B.
S-3: Check compatibility:
· if columns of A ≠ Rows of B, multiplication
S-4: Input elements of both matrices.
S-5: Use 3 nested loops.
· Outer loop for rows of A

. Middle loop for columns of B.
. Inner loop for multiplication and summation.

s-6 : Compute :

$$C[i][j] = c[i][j] + A[i][k] \times B[k][j]$$

s-7 : Display Resultant Matrix

s-8 : STOP

Necessity of Compability :

Matrix multiplication requires multiplying elements of a row from the first matrix with elements of a column from the second matrix.

This is only possible when :

Columns of A = Rows of B

] Discuss the applications of matrix operations in real-world domains such as computer graphics, image processing, machine learning, engineering, and scientific computing.

Matrix operations are widely used in many real-world applications :

1) Computer Graphics :
- Used for rotation, scaling, translation, and projection of objects.
- Helps create 2D & 3D animations and video games.

2) Image processing :
- Images are stored as matrices of pixels.
- Matrix operations are used for filtering, enhancement,

compression, and edge detection.

3) Machine learning
- Data sets are represented as matrices.
- Neural networks perform matrix multiplica
training and prediction.
- Used in AI applications such as speech
recognition.

4) Engineering:
- Used in structural analysis, electrical cir
control systems.
- supports large-scale numerical calculations.

Matrices provide a compact & efficient way to store
large amounts of data, making them essential in
computing and engineering applications.

5) Explain the role of nested loops in implementing
matrix operations in C. Illustrate how nested lo
used for addition, multiplication, and transpose o

ons: Nested loops are fundamental for matrix operatio
matrices contain rows and columns.
- Outer loop processes rows.
- Inner loop processes columns.

Matrix addition :

```
for (i=0; i<r; i++)
{
  for (j=0; j<c; j++)
  {
    sum[i][j] = a[i][j] + b[i][j];
  }
}
```

each element of first matrix is added to the correspondin element of second matrix.

Matrix Multiplication :

```
for (i=0; i<r1; i++)
{
  for (j=0; j<c2; j++)
  {
    result[i][j] = 0;
    for (k=0; k<r1; k++)
    {
      result[i][j] += a[i][k] * b[k][j];
    }
  }
}
```

Three nested loops are required :

First loop → rows of first matrix

Second loop → columns of second matrix

Third loop → multiplication & summation.

Matrix Transpose:

```
for (i=0; i<r; i++)
{
    for (j=0; j<c; j++)
    {
        transpose[j][i] = a[i][j];
    }
}
```

Rows become columns & columns become rows.

∴ Nested loops allow systematic traversal of matrix ele and are essential for performing matrix ~~elements~~ ad subtraction, multiplication, transpose, and other matri related operations in C-programming.

Pseudo Code:

S-① Declare matrices A, B, C, and variables r1, c1, r2, c2.
② Input rows and columns of Matrix A (r1, c1).
③ Input elements of matrix A.
④ Input rows & columns of Matrix B (r2, c2).
⑤ Input elements of Matrix B.

Matrix Addition:

```
If r1 = r2 and c1 = c2 then
for i = 0 to r1-1
    for j = 0 to c1-1
        C[i][j] = A[i][j] + B[i][j]
    end for
end for
```

Display Matrix C
else
Display "Addition not possible"
end if

Matrix Subtraction:
if r1 = r2 and c1 = c2 then
for i = 0 to r1 - 1
for j = 0 to c1 - 1
C[i][j] = A[i][j] - B[i][j]
end for
end for
Display Matrix C
else
Display "Subtraction not possible"
End if

Matrix Multiplication:
if c1 = r2 then
for i = 0 to r1 - 1
for j = 0 to c2 - 1
C[i][j] = c[i][j] + A[i][k] * B[k][j]
end for
end for
end for
Display Matrix C
else
Display "Multiplication not possible"
end if

Transpose of Matrix A :

```
for i = 0 to rl - 1
    for j = 0 to cl - 1
        T[j][i] = A[i][j].
    end for
end for
Display Transpose Matrix T.

End.
STOP
```

16/7/26

Assignment 1

```
#include <stdio.h>

Void input(int m[10][10], int r, int c);
Void display(int m[10][10], int r, int c);
Void add(int a[10][10], int b[10][10], int res[10][10], int r, int c);
Void sub(int a[10][10], int b[10][10], int res[10][10], int r, int c);
Void mul(int a[10][10], int b[10][10], int res[10][10], int r1, int c1, int c2);
Void transpose(int m[10][10], int res[10][10], int r, int c);

Int main()
{
  Int a[10][10], b[10][10], res[10][10];
  Int r1, c1, r2, c2;
  Int choice;
  Int ch;

  Printf("Enter rows and columns of Matrix A: ");
  Scanf("%d%d", &r1, &c1);

  If(r1 < 1 || r1 > 10 || c1 < 1 || c1 > 10)
  {
    Printf("Invalid matrix size! Maximum allowed is 10 x 10.\n");
    Return 0;
  }
```

```
  Printf("Enter elements of Matrix A:\n");
  Input(a, r1, c1);

  Printf("\nEnter rows and columns of Matrix B: ");
  Scanf("%d%d", &r2, &c2);

  If(r2 < 1 || r2 > 10 || c2 < 1 || c2 > 10)
  {
    Printf("Invalid matrix size! Maximum allowed is 10 x 10.\n");
    Return 0;
  }

  Printf("Enter elements of Matrix B:\n");
  Input(b, r2, c2);

  Do
  {
    Printf("\n============================\n");
    Printf(" MATRIX OPERATIONS MENU\n");
    Printf("============================\n");
    Printf("1. Display Matrices\n");
    Printf("2. Addition (A + B)\n");
    Printf("3. Subtraction (A – B)\n");
    Printf("4. Multiplication (A * B)\n");
    Printf("5. Transpose\n");
    Printf("6. Exit\n");
```

```
Printf("==============================\n");
Printf("Enter your choice: ");

If(scanf("%d", &choice) != 1)
{
  Printf("Invalid input! Please enter a number.\n");

  While((ch = getchar()) != '\n' && ch != EOF);

  Choice = 0;
  Continue;
}

Switch(choice)
{
  Case 1:
    Printf("\nMatrix A:\n");
    Display(a, r1, c1);

    Printf("\nMatrix B:\n");
    Display(b, r2, c2);
    Break;

  Case 2:
    If(r1 == r2 && c1 == c2)
    {
```

```
      Add(a, b, res, r1, c1);

      Printf("\nResult of A + B:\n");
      Display(res, r1, c1);
    }
    Else
    {
      Printf("\nAddition not possible. Dimensions must be same.\n");
    }
    Break;

  Case 3:
    If(r1 == r2 && c1 == c2)
    {
      Sub(a, b, res, r1, c1);

      Printf("\nResult of A – B:\n");
      Display(res, r1, c1);
    }
    Else
    {
      Printf("\nSubtraction not possible. Dimensions must be same.\n");
    }
    Break;

  Case 4:
```

```
            If(c1 == r2)
            {
              Mul(a, b, res, r1, c1, c2);

              Printf("\nResult of A * B:\n");
              Display(res, r1, c2);
            }
            Else
            {
              Printf("\nMultiplication not possible.\n");
              Printf("Columns of A must equal Rows of B.\n");
            }
            Break;

        Case 5:
            Printf("\nTranspose of Matrix A:\n");
            Transpose(a, res, r1, c1);
            Display(res, c1, r1);

            Printf("\nTranspose of Matrix B:\n");
            Transpose(b, res, r2, c2);
            Display(res, c2, r2);
            Break;

        Case 6:
            Printf("\nProgram terminated successfully.\n");
```

```
            Break;

        Default:
            Printf("\nInvalid choice! Please enter 1 to 6.\n");
      }

   } while(choice != 6);

   Return 0;
}

Void input(int m[10][10], int r, int c)
{
   Int I, j;

   For(I = 0; I < r; i++)
   {
      For(j = 0; j < c; j++)
      {
         Scanf("%d", &m[i][j]);
      }
   }
}

Void display(int m[10][10], int r, int c)
{
```

```
  Int I, j;

  For(I = 0; I < r; i++)
  {
    For(j = 0; j < c; j++)
    {
      Printf("%d\t", m[i][j]);
    }
    Printf("\n");
  }
}

Void add(int a[10][10], int b[10][10], int res[10][10], int r, int c)
{
  Int I, j;

  For(I = 0; I < r; i++)
  {
    For(j = 0; j < c; j++)
    {
      Res[i][j] = a[i][j] + b[i][j];
    }
  }
}

Void sub(int a[10][10], int b[10][10], int res[10][10], int r, int c)
```

```
{
  Int I, j;

  For(I = 0; I < r; i++)
  {
    For(j = 0; j < c; j++)
    {
      Res[i][j] = a[i][j] – b[i][j];
    }
  }
}

Void mul(int a[10][10], int b[10][10], int res[10][10], int r1, int c1, int c2)
{
  Int I, j, k;

  For(I = 0; I < r1; i++)
  {
    For(j = 0; j < c2; j++)
    {
      Res[i][j] = 0;

      For(k = 0; k < c1; k++)
      {
        Res[i][j] += a[i][k] * b[k][j];
      }
```

```
      }
   }
}

Void transpose(int m[10][10], int res[10][10], int r, int c)
{
   Int I, j;

   For(I = 0; I < r; i++)
   {
      For(j = 0; j < c; j++)
      {
         Res[j][i] = m[i][j];
      }
   }
}
```